

Making Your Code Walk: GNU Autotools



Have you ever wondered why so many files are shipped in a software source tarball for Linux? Or dreamt of making your own newly developed package highly portable across various Linux platforms? This article gives you a basic idea of how package makers/maintainers port their code to various platforms without too many changes to their code.

You've probably downloaded various package sources from the Internet and compiled (built) them when required to do so. The common steps are `./configure`, `make` and `sudo make install`. If you just need to use a package, then these steps are usually sufficient—but it's different if you are a package developer/maintainer. Besides designing and coding to develop the program, you need it to be able to port packages to various Linux-like platforms. Trust me: sometimes the second task is far tougher, if not handled properly.

Portability and GNU Autotools

Before going into the details, what is 'portability'? Here's one possible definition: if you can write your code so that it can be run on a different environment without much effort, then your code is portable. There are factors that reduce code portability, like the differences in processors, operating systems, preprocessors, compilers, etc. Moreover, floating point issues, availability of dynamic libraries, differences in file-systems, etc, make portability a much bigger challenge. Linux supports different architectures. Sometimes it's not possible to run

a binary copied from another Linux machine; it needs to be compiled/built and installed on the target PC—which sometimes requires more than just *configure*, *make* and *make install*.

Here's where GNU Autotools comes in; it may increase your projects' portability many times over—even to systems on which you have never run and tested your code. So, if you are planning to write an open source package targeting the UNIX/Linux platform, your best choice is to use GNU Autotools. If you're only targeting Windows and will never need to port to Linux/UNIX in the future, then you can safely refrain from using Autotools.

GNU Autotools offers three main tools:

- Autoconf, which is used to create the *configure* script automatically.
- Automake, which is used to create flexible and consistent *Makefiles*.
- Libtool, which helps create portable shared libraries.

Here, I will focus only on Autoconf and Automake.

Autoconf provides native support for C, C++, Objective C, Fortran, etc. Native support for Java is still not available in Autotools.